SpThreshold: Replication Threshold for Bayesian Multilevel Models of Areal Data

Part I: Replication threshold via m_star()

[1] Load the package and set the seed:

```
library(SpThreshold)
set.seed(2026)
```

[2] Generate a random connected adjacency matrix:

```
n <- 100
W <- create_random_W(n_locs = n)
```

[3] Set the residual variance, nonspatial random-intercept variance, and spatial correlation. The spatial variance scale is obtained through variance calibration:

```
sigma2 <- 0.50
tau_ns_2 <- 0.50
rho <- 0.70

cal <- variance_calibration(
  W = W,
  rho = rho,
  tau_ns_2 = tau_ns_2
)

data.frame(
  tau_ns_2 = cal$tau_ns_2,
  tau_sp_2 = cal$tau_sp_2
)
```

```
##   tau_ns_2  tau_sp_2
## 1      0.5 0.4509499
```

[4] Compute the approximate threshold under three covariate structures. Covariates are centered and standardized to population variance one.

- C1: Independent covariate values, with most variation within areas:

```
m_obs <- 20
N <- n * m_obs
loc <- rep(1:n, each = m_obs)

X_c1 <- rnorm(N)
X_c1 <- (X_c1 - mean(X_c1)) /
  sqrt(mean((X_c1 - mean(X_c1))^2))

res_c1 <- m_star(
  W = W,
```

```
  sigma2 = sigma2,
  rho = rho,
  tau_ns_2 = tau_ns_2,
  X = X_c1,
  loc = loc,
  method = "approx"
)

res_c1$m_star
```

```
## beta1
##     2
```

- C2: Covariate variation both within and between areas:

```
loc_means <- rnorm(n)
X_c2 <- rnorm(N, mean = loc_means[loc], sd = 1)
X_c2 <- (X_c2 - mean(X_c2)) /
  sqrt(mean((X_c2 - mean(X_c2))^2))

res_c2 <- m_star(
  W = W,
  sigma2 = sigma2,
  rho = rho,
  tau_ns_2 = tau_ns_2,
  X = X_c2,
  loc = loc,
  method = "approx"
)

res_c2$m_star
```

```
## beta1
##    18
```

- C3: Covariate variation only between areas:

```
X_c3 <- loc_means[loc]
X_c3 <- (X_c3 - mean(X_c3)) /
  sqrt(mean((X_c3 - mean(X_c3))^2))

res_c3 <- m_star(
  W = W,
  sigma2 = sigma2,
  rho = rho,
  tau_ns_2 = tau_ns_2,
  X = X_c3,
  loc = loc,
  method = "approx"
)

res_c3$m_star
```

```
## beta1
##   Inf
```

[5] Compare the approximate and exact thresholds under C2. The exact threshold identifies the replication level at which the full conditional variance difference remains within tolerance for every larger sample size:

```
res_c2_exact <- m_star(
  W = W,
  sigma2 = sigma2,
  rho = rho,
  tau_ns_2 = tau_ns_2,
  X = X_c2,
  loc = loc,
  method = "exact"
)

data.frame(
  m_approx = res_c2_exact$m_approx,
  m_exact = res_c2_exact$m_exact
)
```

```
##       m_approx m_exact
## beta1       18      10
```

[6] Vary ρ under C2, holding τ^2_{ns} and σ^2 fixed:

```
rho_seq <- c(0.05, 0.25, 0.50, 0.75, 0.95)

m_star_rho <- sapply(rho_seq, function(r) {
  m_star(
    W = W,
    sigma2 = sigma2,
    rho = r,
    tau_ns_2 = tau_ns_2,
    X = X_c2,
    loc = loc
  )$m_star
})

data.frame(
  rho = rho_seq,
  m_approx = m_star_rho
)
```

```
##    rho m_approx
## 1 0.05        2
## 2 0.25        3
## 3 0.50        9
## 4 0.75       22
## 5 0.95       61
```

[7] Vary $\kappa = \tau^2_{\text{ns}}/\sigma^2$ under C2:

```r
k_seq <- c(0.05 / 0.95, 0.50 / 0.50, 0.95 / 0.05)

m_star_k <- sapply(k_seq, function(k) {
  m_star(
    W = W,
    sigma2 = 1 / (1 + k),
    rho = rho,
    tau_ns_2 = k / (1 + k),
    X = X_c2,
    loc = loc
  )$m_star
})

data.frame(
  kappa = k_seq,
  m_approx = m_star_k
)
```

```
##         kappa m_approx
## 1  0.05263158      335
## 2  1.00000000       18
## 3 19.00000000        2
```

[8] Vary the tolerance γ under C2:

```r
gamma_seq <- c(0.01, 0.05, 0.10, 0.20)

m_star_gamma <- sapply(gamma_seq, function(g) {
  m_star(
    W = W,
    sigma2 = sigma2,
    rho = rho,
    tau_ns_2 = tau_ns_2,
    X = X_c2,
    loc = loc,
    gamma = g
  )$m_star
})

data.frame(
  gamma = gamma_seq,
  m_approx = m_star_gamma
)
```

```
##   gamma m_approx
## 1  0.01       88
## 2  0.05       18
## 3  0.10        9
## 4  0.20        5
```

[9] An adjacency matrix can also be constructed from a shapefile. This example uses the North Carolina shapefile included in `sf` and requires the `sf` and `spdep` packages:

```r
library(sf)
```

```
## Warning: package 'sf' was built under R version 4.4.3
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```r
shp_path <- system.file("shape/nc.shp", package = "sf")
polygons <- st_read(shp_path, quiet = TRUE)
W_nc <- create_W_from_shapefile(polygons)

dim(W_nc)
```

```
## [1] 100 100
```

Part II: Model fitting via spfit()

The function `spfit()` fits spatial or nonspatial multilevel models using MCMC.

[1] Generate a Gaussian dataset with covariate variation within and between areas:

```r
set.seed(2026)

n_locs <- 50
m_per <- 10
N <- n_locs * m_per

W_small <- create_random_W(n_locs)
loc_small <- rep(1:n_locs, each = m_per)

beta_true <- c(0.50, 1.50)
sigma2_true <- 0.50
tau_sp_2_true <- 0.50
rho_true <- 0.50

D <- diag(rowSums(W_small))
Q_true <- rho_true * (D - W_small) +
  (1 - rho_true) * diag(n_locs)

theta_true <- as.numeric(
  t(chol(tau_sp_2_true * solve(Q_true))) %*% rnorm(n_locs)
)
theta_true <- theta_true - mean(theta_true)

loc_means_x <- rnorm(n_locs)
x_small <- rnorm(N, mean = loc_means_x[loc_small], sd = 1)
x_small <- (x_small - mean(x_small)) /
  sqrt(mean((x_small - mean(x_small))^2))

X_small <- cbind(1, x_small)
colnames(X_small) <- c("(Intercept)", "x")
```

```r
y_small <- as.numeric(
  X_small %*% beta_true + theta_true[loc_small]
) + rnorm(N, sd = sqrt(sigma2_true))
```

[2] Fit the spatial model:

```r
fit_sp <- spfit(
  y = y_small,
  X = X_small,
  loc = loc_small,
  W = W_small,
  model_indicator = 1,
  mcmc_samples = 15000,
  burnin = 5000
)
```

[3] Summarize the posterior after burn-in:

```r
summary_sp <- posterior_summary(fit_sp)

summary_sp[, c(
  "parameter", "mean", "median", "variance", "lower", "upper"
)]
```

```
##     parameter      mean    median     variance     lower     upper
## 1 (Intercept) 0.4839229 0.4835400 0.0009955922 0.4218503 0.5463042
## 2           x 1.5117364 1.5110659 0.0018307677 1.4286305 1.5965659
## 3      sigma2 0.5040606 0.5023834 0.0011196203 0.4418053 0.5737497
## 4        tau2 0.5606238 0.5427661 0.0185115278 0.3473775 0.8759171
## 5         rho 0.6747627 0.6898134 0.0282214681 0.3160114 0.9467907
```

[4] Fit the nonspatial model to the same data:

```r
fit_ns <- spfit(
  y = y_small,
  X = X_small,
  loc = loc_small,
  W = W_small,
  model_indicator = 0,
  mcmc_samples = 15000,
  burnin = 5000
)

summary_ns <- posterior_summary(fit_ns)

summary_ns[, c(
  "parameter", "mean", "median", "variance", "lower", "upper"
)]
```

```
##     parameter      mean    median     variance     lower     upper
## 1 (Intercept) 0.4837999 0.4834230 0.0009999098 0.4218775 0.5453365
## 2           x 1.5130018 1.5130064 0.0018861184 1.4282938 1.5974053
## 3      sigma2 0.5051031 0.5033045 0.0011348092 0.4442397 0.5754823
## 4        tau2 0.6197156 0.6024017 0.0200077350 0.3934491 0.9446453
```

[5] Compare the marginal posterior variances of the covariate coefficient using the retained MCMC draws:

```
draws_sp <- posterior_draws(fit_sp)
draws_ns <- posterior_draws(fit_ns)

var_sp <- var(draws_sp$x)
var_ns <- var(draws_ns$x)
rel_diff <- abs(var_sp - var_ns) / var_ns

data.frame(
  variance_spatial = var_sp,
  variance_nonspatial = var_ns,
  relative_difference = rel_diff,
  gamma = 0.05,
  within_tolerance = rel_diff <= 0.05
)
```

```
##   variance_spatial variance_nonspatial relative_difference gamma
## 1      0.001830768         0.001886118          0.02934635  0.05
##   within_tolerance
## 1             TRUE
```

[6] Compute the approximate and exact thresholds using the generating spatial covariance parameters and the calibrated nonspatial variance. These thresholds describe the full conditional variance comparison at fixed covariance parameters:

```
res <- m_star(
  W = W_small,
  sigma2 = sigma2_true,
  rho = rho_true,
  tau_sp_2 = tau_sp_2_true,
  X = x_small,
  loc = loc_small,
  method = "exact"
)

data.frame(
  m_actual = m_per,
  m_approx = res$m_approx,
  m_exact = res$m_exact
)
```

```
##       m_actual m_approx m_exact
## beta1       10        7       2
```

[7] The data-frame and matrix interfaces give the same coefficient draws when supplied with the same data and random seed:

```
df_small <- data.frame(
  y = y_small,
  x = x_small,
  area = loc_small
```

```
)

set.seed(7)
fit_df <- spfit(
  data = df_small,
  formula = y ~ x,
  area = "area",
  W = W_small,
  model_indicator = 1,
  mcmc_samples = 15000,
  burnin = 5000
)

set.seed(7)
fit_vec <- spfit(
  y = y_small,
  X = X_small,
  loc = loc_small,
  W = W_small,
  model_indicator = 1,
  mcmc_samples = 15000,
  burnin = 5000
)

identical(fit_df$beta, fit_vec$beta)
```

```
## [1] TRUE
```

Part III: Two-stage procedure

Apply the two-stage procedure to the same dataset, treating the covariance parameters as unknown.

[1] Estimate the covariance parameters:

```
two_stage <- two_stage_estimate(
  y = y_small,
  X = X_small,
  loc = loc_small,
  W = W_small
)

two_stage$estimates
```

```
##         rho  tau_ns_2  tau_sp_2    sigma2 upper_boundary
## 1 0.7172861 0.5920854 0.5191862 0.5022298          FALSE
```

[2] Examine the full conditional variance comparison at the observed design:

```
two_stage$conditional_variances
```

```
##   coefficient spatial_variance iid_variance conditional_gap
## 1           x      0.001805741  0.001870952      0.03485454
```